Flutter

Séance 1 · Pourquoi Flutter, installation, Dart

Lancer le téléchargement maintenant

L'archive fait **1,6 Go**. Lancez le téléchargement **maintenant**, avant tout le reste.

```
storage.googleapis.com/flutter_infra_release/releases/stable/
linux/flutter_linux_3.47.8-stable.tar.xz
```

Le cours de la prochaine heure se suit sans machine.

> Partage de connexion mobile ou câble Ethernet : à utiliser sans hésiter, ça soulage le Wi-Fi de la salle pour tout le monde.

12 séances, une seule application

- **12 séances de 4 h**, une application unique construite du début à la fin
- **Pantry**, gestion de garde-manger, sur l'API Open Food Facts
- Cible **Chrome** jusqu'à la séance 7, **Android** à partir de la séance 8
- Structure d'une séance : deux blocs de cours, deux TP, un débrief

Évaluation, 80 % un rendu final en équipe, travaillé pendant les séances 11 et 12 ; 20 % de contrôle continu sur les livrables de TP. Le rendu des TP est expliqué en séance 2, le rendu final en séance 6.

Hors du cours : stores, backend, iOS

- **CI/CD, signature, publication sur les stores**, survolés en séance 10, jamais pratiqués
- **Backend**, on consomme des API, on n'en écrit pas
- **iOS**, démonstration éventuelle, exclu des rendus
- **Desktop**, optionnel, selon les postes
- Flutter embarqué, shaders, animations très avancées, monorepos

Le temps correspondant est consacré à l'architecture, la gestion d'état et les tests.

Aujourd'hui

Horaire	Bloc
0:00 – 0:20	Lancement des téléchargements
0:20 – 1:05	Pourquoi Flutter
1:05 – 2:00	Installation en autonomie
2:00 – 2:15	Pause
2:15 – 3:15	Dart en accéléré, sur DartPad
3:15 – 4:10	Tutoriel officiel, étapes 3 à 8
4:10 – 4:15	Consignes

Objectif unique de la séance : `flutter run -d chrome` **fonctionne**.

Pourquoi Flutter

Le paysage, l'architecture, les limites

Quatre façons de faire une application mobile

Approche	Langage	Rendu
Natif	Kotlin / Swift	composants du système
React Native	TypeScript	composants du système, pilotés depuis JS
KMP + Compose Multiplatform	Kotlin	Skia, ou natif selon la cible
Flutter	Dart	moteur de rendu propre

La dernière colonne détermine la majorité des différences qui suivent.

Natif : deux bases de code

Avantages, accès immédiat aux nouveautés du système, outillage mature, performances de référence, conformité visuelle par construction.

Limites, deux bases de code, deux équipes, deux calendriers. Chaque fonctionnalité est écrite deux fois et diverge.

Choix par défaut lorsque l'application est fortement intégrée au système, ou qu'une seule plateforme est visée.

React Native : du JavaScript qui pilote le natif

Du JavaScript qui pilote de **vrais composants du système**.

Avantages, écosystème JavaScript, hot reload, réutilisation des compétences web, rendu natif sans travail supplémentaire.

Limites, la frontière entre JavaScript et natif reste un point sensible malgré la nouvelle architecture, et les performances dépendent de son usage. Écosystème de bibliothèques vaste mais inégal.

KMP : partager la logique, garder l'interface

Kotlin Multiplatform partage la logique métier ; l'interface reste native, ou devient partagée avec **Compose Multiplatform**.

Avantages, adoption progressive, un module partagé à la fois. Langage et modèle d'interface déjà connus.

Limites, maturité inégale selon les cibles, outillage plus récent, communauté plus restreinte sur la partie interface partagée.

> Alternative la plus proche de Flutter pour un développeur venant de Compose.

Flutter dessine chaque pixel

Dart compilé en **AOT**, et une interface **dessinée** par le moteur.

Flutter ne demande pas au système de créer un bouton : il dessine le bouton, pixel par pixel, avec son propre moteur de rendu (**Impeller**).

Une seule base de code pour Android, iOS, web, Windows, macOS et Linux.

Aucun pont entre Dart et l'écran



Pas de traduction d'appels à la frontière, pas de sérialisation à chaque interaction.

En contrepartie : rien n'est fourni par le système. Tout ce que vous voyez à l'écran a été dessiné par Flutter, y compris les barres de défilement et les curseurs de texte.

Un rendu identique partout

- **Rendu identique sur tous les appareils.** Les écarts entre surcouches constructeur disparaissent.
- **Le hot reload préserve l'état** de l'application entre deux modifications du code.
- **Les composants suivent le calendrier de Flutter**, pas celui du système.
- **Tout est widget** : mise en page, marge, style, animation. Une seule notion à assimiler.

Le prix : poids, intégration, rendu non natif

- **Poids du binaire.** Le moteur de rendu est embarqué : plusieurs Mo incompressibles, y compris pour une application vide.
- **Intégration native.** Tout accès à une API système passe par un plugin, existant ou à écrire. Écriture d'un plugin en séance 8.
- **Rendu non conforme par défaut.** Material sur iOS ne suit pas les conventions du système ; l'adaptation est à la charge du développeur.
- **Gouvernance.** La feuille de route dépend d'un seul éditeur.
- **Stabilité de l'écosystème.** Les paquets tiers suivent inégalement les versions. Un tutoriel de plus de 18 mois est fréquemment obsolète.

Les quatre approches répondent à des contraintes différentes

Le choix de Flutter pour ce cours est pédagogique, et ne constitue pas un classement.

La stack du cours

Domaine	Choix
État	`setState` puis **`flutter_bloc`**
Injection	**`get_it`** puis **`injectable`**
Navigation	`Navigator` puis **`go_router`**
Réseau	**`dio`**
Modèles	**`freezed`** + `json_serializable`
Base locale	**`drift`**

Correspondances avec l'écosystème Android : `Bloc` ≈ `ViewModel`, `State` ≈ `StateFlow`, `get_it` ≈ Koin, `injectable` ≈ Hilt, `drift` ≈ Room.

Installation

Objectif : `flutter run -d chrome`

Chrome d'abord : rien d'autre à installer

`flutter run -d chrome` ne demande **ni SDK Android, ni Gradle, ni licences, ni émulateur, ni droits administrateur**.

C'est la seule cible installable en une heure sur les postes de la salle.

Android est introduit en séance 8, les autres plateformes en séance 9.

Installer par l'archive, en 3.47.8

1. **Par l'archive, jamais par `git clone`.** Le clone échoue sur ces postes : le SDK ne parvient plus à déterminer sa propre version. Dans VS Code, *Locate SDK*, jamais *Download SDK*, qui clone.
2. **Version gelée : 3.47.8.** `flutter upgrade` est inopérant avec une installation par archive.
3. **`PATH` dans `~/.bashrc`**, pas dans `/etc` : les droits root ne sont pas disponibles.

Le détail est dans `docs/installation.md`, sur le dépôt du cours. La référence reste **docs.flutter.dev/install/manual**, choix **Linux**, avec l'archive 3.47.8 du tableau : le bouton de la page sert la dernière stable.

Seule la ligne Chrome doit être verte

L'outil vérifie toutes les cibles possibles, y compris celles qui ne servent pas aujourd'hui.

Ligne	Statut
`Chrome`	**doit être verte**
`Android toolchain`	présente sur les postes de la salle, absente ailleurs · sans effet aujourd'hui
`Linux toolchain`	desktop optionnel, sans effet
`Android Studio`	non utilisé, VS Code suffit

Une seule ligne conditionne la suite de la séance : `Chrome`.

Pause

15 minutes

Dart

Écarts avec Kotlin

Quatre explications, des exercices sur DartPad

Quatre explications courtes, chacune suivie d'exercices sur **dartpad.dev**. Aucune installation, tout s'exécute dans le navigateur.

L'asynchrone occupe le bloc le plus long : c'est le point le moins maîtrisé à l'entrée, et il reviendra en séances 4 et 5.

Sur le dépôt du cours : `docs/seance-01-dart-exercices.md` pour les énoncés, `docs/dart-pour-devs-kotlin.md` pour les correspondances Kotlin ↔ Dart, à garder ouvert.

Un poste dont l'installation de Flutter n'est pas terminée peut suivre ce bloc entièrement.

Les exercices non terminés se reprennent pendant le tutoriel, qui est auto-rythmé.

Dart ressemble à Kotlin, sauf l'asynchrone

- Typage statique, inférence, **null safety**, comme Kotlin
- Compilation **AOT** en production, **JIT** en développement : c'est cette dernière qui permet le hot reload
- **Un seul thread** par isolate, avec une event loop
- Pas de coroutines : `Future` et `Stream`

Le reste se transpose directement. Les écarts sont peu nombreux mais réguliers.

Le type se place avant le nom

```kotlin
fun somme(a: Int, b: Int): Int = a + b        // Kotlin
```

```dart
int somme(int a, int b) => a + b;             // Dart
```

Différence de syntaxe la plus visible au début.

Null safety : presque comme Kotlin

Kotlin	Dart
`String?`	`String?`
`x!!`	`x!` · un seul point d'exclamation
`x ?: y`	`x ?? y`
`lateinit var`	`late` · y compris sur un `final`

```dart
late final String valeur = calculCouteux();  // évalué au premier accès
```

`late` permet en outre l'initialisation lazy, ce que `lateinit` ne fait pas.

`const` : un widget jamais reconstruit

Mot-clé	Signification
`final`	assigné une fois, **valeur calculée à l'exécution**
`const`	valeur connue **à la compilation**

```
final maintenant = DateTime.now();   // ok
const pi = 3.14159;                  // ok
```

Deux objets `const` identiques sont **le même objet en mémoire**.

C'est ce qui rend `const` important en Flutter : **un widget `const` n'est pas reconstruit**.

`identical(const [1,2], const [1,2])` renvoie `true`. À exécuter en direct.

Exercice 1

null safety, 5 minutes

`docs/seance-01-dart-exercices.md`

Un `await` oublié ne lève aucune erreur

Kotlin	Dart
`suspend fun`	`Future<T>` + `async`
`Flow<T>`	`Stream<T>`
`collect { }`	`await for`
structured concurrency	**rien d'équivalent**

```dart
Future<String> charger() async {
  await Future.delayed(const Duration(seconds: 1));
  return 'chargé';
}
```

Une fonction `async` renvoie **toujours** un `Future`. L'oubli du `await` ne provoque aucune erreur de compilation ni d'exécution : c'est la source d'erreur la plus fréquente.

Un thread, deux files

Un seul thread, **deux files**, et la première est prioritaire.

File	Alimentée par	Traitement
microtask	`scheduleMicrotask`, `.then` sur un `Future` déjà complété	vidée **entièrement**
event	`Future(…)`, E/S, timers, gestes	**une seule** par tour

```dart
print('1');
Future(() => print('2'));
Future.microtask(() => print('3'));
print('4');
```

Affiche `1 4 3 2`. Le synchrone d'abord, puis toutes les microtasks, puis un événement.

Une microtask qui en planifie une autre sans fin **bloque la file d'événements**, donc l'interface.

Exercices 2, 3 et 4

Boucle d'événements · `Future` · `Stream`, 15 minutes

Sur l'exercice 3 : retirer le `await`, et regarder ce qui s'affiche.

`map` et `where` sont lazy

`map` et `where` sont **lazy**. Sans `.toList()`, rien n'est calculé.

```
final widgets = [
  const Entete(),
  if (afficherTitre) const Titre(),
  ...autresWidgets,
  for (final p in produits) Ligne(p),
];
```

Collection-if, spread et boucles **dans un littéral de liste** : cette syntaxe remplace les `if` et les boucles lors de la construction d'interface. Usage constant à partir de la séance 2.

Exercice 5

collection-if et spread, 5 minutes

Le compilateur vérifie chaque cas

```
sealed class Resultat {}
class Succes extends Resultat { Succes(this.donnees); final String donnees; }
class Echec  extends Resultat { Echec(this.message);  final String message; }

String decrire(Resultat r) => switch (r) {
  Succes(:final donnees) => 'ok : $donnees',
  Echec(:final message)  => 'erreur : $message',
};
```

La compilation échoue si un cas manque.

Cette structure servira à modéliser les quatre états d'interface (chargement, données, erreur, vide) à partir de la séance 4.

Records : plusieurs valeurs sans classe

```dart
({double lat, double lon}) position() => (lat: 48.85, lon: 2.35);

final p = position();
print(p.lat);
```

Plusieurs valeurs de retour sans déclaration de classe. Kotlin ne dispose que de `Pair` et `Triple`.

Pour une donnée circulant dans l'application, une classe reste préférable : un record n'a pas de nom, donc pas de sens métier.

Exercice 6

`sealed` et `switch` exhaustif, 10 minutes

Une fois qu'il fonctionne : retirer un cas du `switch`, et lire l'erreur.

Absent de Dart

Référence, figure aussi dans la table Kotlin ↔ Dart.

Kotlin	Dart
`let`, `apply`, `run`, `also`	la cascade `..`, partiellement
`data class`	classe ordinaire, ou `freezed`
`object` / `companion object`	variable de premier niveau, ou `static`
coroutines, structured concurrency	`Future` / `Stream`, sans annulation
`internal`	`_` · privé **au fichier**, pas à la classe

Le préfixe `_` rend privé **au fichier**, et non à la classe. C'est ce qui permet à un `_XxxState` d'accéder aux champs privés de la classe voisine.

Tutoriel officiel · étapes 3 à 8

55 minutes · `docs.flutter.dev/learn/pathway/tutorial`

De *Create a Flutter app* à *Learn about stateful widgets*, en autonomie, sur Chrome. Aucun paquet à ajouter.

Garder `import 'package:flutter/material.dart';` : l'étape 4 affiche `package:material_ui`, absent du projet en 3.47.8. N'ajouter que `import 'game.dart';`.

Étapes non terminées : à finir après la séance. Étapes 9 et suivantes : facultatives.

Travail à faire

- [ ] Terminer l'installation : `flutter run -d chrome` doit fonctionner
- [ ] Parcourir le catalogue de widgets, et regarder trois ou quatre *Widget of the Week*. C'est le pointeur le plus rentable avant la séance 2, qui porte sur la mise en page
- [ ] Étapes 3 à 8 du tutoriel, si elles ne sont pas terminées

Séance 2 : widgets, mise en page, état local, formulaires. Début du développement de Pantry.

Questions

Pointeurs

Ressource	Adresse
Dépôt du cours	`github.com/mduruisseau/pantry-2026` → `docs/`
Installation	`docs.flutter.dev/install/manual` → Linux
Archive 3.47.8	`storage.googleapis.com/flutter_infra_release/releases/stable/linux/`
Tutoriel	`docs.flutter.dev/learn/pathway/tutorial` · étapes 3 à 8
DartPad	`dartpad.dev`
Langage Dart	`dart.dev/language`
Catalogue de widgets	`docs.flutter.dev/ui/widgets`
Cookbook et ressources	`docs.flutter.dev/reference/learning-resources`
Samples	`flutter.github.io/samples`
Widget of the Week	chaîne Flutter sur YouTube
Référence de l'API	`api.flutter.dev`